



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

DIGITAL ASSIGNMENT 4

Name: Soumyadeep Ganguly

Reg No: 24MDT0082

Course: Big Data Analytics Lab

Course Code: PMDS507P

Explain the procedure to run the Hadoop WordCount program using the in-built mapreduce-examples.jar.

Step 1: Start Hadoop Services

First, we start the necessary Hadoop daemons for both the Distributed File System (DFS) and the resource manager (YARN).

```
hduser@sjt217score010:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [sjt217score010]
2025-10-03 12:14:09,292 WARN util.NativeCodeLoader: Unable to load native-hadoop
library for your platform... using builtin-java classes where applicable
hduser@sjt217score010:~$
```

```
hduser@sjt217score010:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hduser@sjt217score010:~$
```

Step 2: Verify Daemons are Running

We use the jps (Java Virtual Machine Process Status) command to confirm that all required Hadoop processes have started successfully

```
hduser@sjt217score010:~$ jps
12883 ResourceManager
12102 DataNode
13512 Jps
13048 NodeManager
12446 SecondaryNameNode
11886 NameNode
hduser@sjt217score010:~$
```

Step 3: Create an Input Directory in HDFS

Next, we create a directory named /input within the Hadoop Distributed File System (HDFS) to store our source text file.

```
hduser@sjt217score010:~$ hdfs dfs -mkdir -p /input
2025-10-03 12:28:36,464 WARN util.NativeCodeLoader: Unable to load native
ry for your platform... using builtin-java classes where applicable
```

Step 4: Upload the Input File to HDFS

We then copy the text file IntroToHadoop.txt from the local filesystem to the newly created /input directory in HDFS.

```
hduser@sjt217score010:~$ hdfs dfs -put /home/hduser/Desktop/OperationHadoop/IntroToHad
oop.txt /input/
2025-10-03 12:29:05,046 WARN util.NativeCodeLoader: Unable to load native-hadoop libra
ry for your platform... using builtin-java classes where applicable
```

Step 5: Verify File Upload and Content

We list the contents of the HDFS /input directory to confirm the file exists and then use the cat command to display its content, ensuring it was uploaded correctly.

```
hduser@sjt217score010:~$ hdfs dfs -ls /input/
2025-10-03 12:29:18,173 WARN util.NativeCodeLoader: Unable to load native-hadoop libra
ry for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--  1 hduser supergroup      187 2025-10-03 12:29 /input/IntroToHadoop.txt
```

Step 6: View file contents:

Using -cat command we can view the file contents

```
hduser@sjt217score010:~$ hdfs dfs -cat /input/IntroToHadoop.txt
2025-10-03 12:30:48,127 WARN util.NativeCodeLoader: Unable to load native-hadoop libra
ry for your platform... using builtin-java classes where applicable
Hadoop is an open-source framework that handles the storage and processing of massive
datasets across clusters of inexpensive computers rather than relying on a single expen
sive machine.
hduser@sjt217score010:~$
```

Step 7: Execute the WordCount MapReduce Job using JAR file

Now we execute the WordCount program using the `hadoop jar` command, specifying the example JAR file, the class `wordcount`, the HDFS input path `/input`, and the HDFS output path `/output`.

```
hduser@sjt217score010: $ hadoop jar /home/hduser/hadoop/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.1.jar wordcount /input /output
2025-10-03 12:51:10,296 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2025-10-03 12:51:11,558 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2025-10-03 12:51:11,781 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2025-10-03 12:51:11,781 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2025-10-03 12:51:12,322 INFO input.FileInputFormat: Total input files to process : 1
2025-10-03 12:51:12,425 INFO mapreduce.JobSubmitter: number of splits:1
2025-10-03 12:51:12,692 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local1093694298_0001
2025-10-03 12:51:12,692 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-10-03 12:51:12,947 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2025-10-03 12:51:12,948 INFO mapreduce.Job: Running job: job_local1093694298_0001
2025-10-03 12:51:12,952 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2025-10-03 12:51:12,970 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2025-10-03 12:51:12,970 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup_temporary folders under output directory:false, ignore cleanup failures: f
2025-10-03 12:51:12,972 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
```

Step 8: View the Final Output

Finally, we use the `cat` command to view the contents of the output file `part-r-00000` created by the MapReduce job in the `/output` directory, which contains the final word counts.

```
hduser@sjt217score010: $ hdfs dfs -cat /output/part-r-00000
2025-10-03 12:57:17,083 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Hadoop 1
a 1
across 1
an 1
and 1
clusters 1
computers 1
datasets 1
expensive 1
framework 1
handles 1
inexpensive 1
is 1
machine 1
massive 1
of 2
on 1
open-source 1
processing 1
rather 1
relying 1
single 1
storage 1
than 1
that 1
the 1
```